Agentic Systems

Institute: Indian Institute of Information Technology (IIIT) Surat
Participants: Senior Undergrads.
Lab: deepdynamicsai.github.io **Coordinator:** Dr. Pradeep Singh
Lecture 1

Why Begin with Agents Before LLMs?

The word *agent* is currently used very broadly. A chatbot that answers one prompt is sometimes called an agent; a deterministic script with several API calls is also called an agent; and a long-running autonomous system interacting with external tools may receive the same label. For a rigorous study of agentic systems, we need a definition that is independent of any particular software framework or language model.

The central idea is simple: an *agent* is a computational system that repeatedly receives information about an environment, maintains whatever internal information it needs, chooses an action, and thereby influences what happens next. The key word is *repeatedly*. An agent is naturally a closed-loop system: its later decisions depend on observations produced after earlier actions.

A useful first abstraction is

observe $\longrightarrow$ update internal state $\longrightarrow$ choose action $\longrightarrow$ act $\longrightarrow$ observe again.

Large language models are important because they can implement part of the decision-making mechanism inside this loop, but the loop itself is conceptually older and more general than LLMs. In this course, we therefore study the agent structure first and add LLMs later.

Learning Objectives

After this lecture, a student should be able to:

- distinguish an agent from a one-shot program, a chatbot, a fixed workflow, and a retrieval system;
- formally identify the environment state, observation, action space, memory, policy, objective, and stopping rule of a simple agent;
- explain the difference between the *true state of the world* and the *information available to the agent*;
- describe deterministic and stochastic environments and fully and partially observable settings;
- explain why memory is needed when the current observation does not contain enough information for good decision making;
- interpret an agent as a closed-loop dynamical system;
- formulate the unknown-function scientist problem as a complete agent–environment interaction;
- identify which parts of an agent can later be replaced by an LLM and which parts should remain under explicit program control.

Ordinary Programs to Agents

Consider an ordinary function $y = f(x)$. The input x is supplied, the program computes $f(x)$, returns y, and terminates. There is no persistent interaction with an external world.

Now consider a program that is asked to diagnose a faulty server. It may first call `ping_server`, inspect the result, then decide whether to inspect memory, disk usage, logs, or configuration. The second action depends on the first observation. The program may stop early if the evidence is already decisive. This is qualitatively different from evaluating a single function.

An agent can therefore be viewed as implementing a sequence $o_1, a_1, o_2, a_2, \ldots, o_T, a_T$, where the action a_t is chosen using information available up to time t. The interaction history before choosing a_t may be written as $h_t = (o_1, a_1, o_2, a_2, \ldots, o_t)$.

The most general deterministic policy can be written as $a_t = \pi(h_t)$. In practice, carrying the entire raw history is often unnecessary, so the agent maintains an internal memory M_t that summarizes the relevant past and uses $a_t = \pi(M_t, o_t)$.

This shift from a fixed mapping $x \mapsto y$ to a sequential policy $h_t \mapsto a_t$ is one of the most important conceptual steps in agentic systems.

The Formal Agent–Environment Model

We now introduce a minimal formal model. Time is discrete: $t = 0, 1, 2, \ldots$.

4.1 Environment State

Let $s_t \in \mathcal{S}$ denote the true state of the environment at time t. The state contains whatever information would be sufficient, in principle, to describe the world relevant to the task.

For a grid-navigation problem, s_t may contain the agent's location, obstacle locations, battery level, and goal location. For the unknown-function scientist problem, s_t may include the hidden function f together with the current experiment count. For server diagnosis, s_t may include the true hidden fault.

The environment state is not necessarily visible to the agent.

4.2 Observation

The agent receives an observation $o_t \in \mathcal{O}$. The observation is the information made available to the agent about the environment.

In a fully observable setting, o_t may determine s_t completely. In a partially observable setting, different hidden states may produce the same observation. Thus, in general, $o_t \neq s_t$.

This distinction is fundamental. A server may truly suffer from memory exhaustion, but the agent does not directly observe the label `memory_exhaustion`. It only sees indirect evidence such as memory usage, service failures, or log messages.

4.3 Actions

At time t, the agent chooses an action $a_t \in \mathcal{A}_t$. The action set may depend on the current situation. For example, if a diagnostic tool has already been used and tools are single-use, it may no longer belong to $\mathcal{A}_t$.

Actions may directly modify the physical or computational environment, such as moving a robot, editing a file, calling an API, sending a message, or executing code. Other actions may primarily gather information, such as inspecting a log file or running a diagnostic test.

This gives an important distinction:

- *instrumental actions* change the world to move toward the objective;
- *information-gathering actions* are chosen primarily because their observations may improve future decisions.

Many agentic systems require both.

4.4 Environment Dynamics

The environment changes after an action. In a deterministic environment, the next state may be represented by $s_{t+1} = F(s_t, a_t)$, for some transition function F.

In a stochastic environment, the next state is random and is described by a transition distribution $P(s_{t+1} \mid s_t, a_t)$. The same action taken in the same state may then produce different outcomes on different runs.

The observation may similarly be deterministic, $o_t = G(s_t)$, or stochastic, described by $P(o_t \mid s_t)$.

4.5 Memory

The agent may maintain internal memory M_t. A general memory update is $M_{t+1} = U(M_t, o_{t+1}, a_t)$, where U is a memory-update rule.

Memory is useful because the latest observation alone may not contain enough information to choose a good action. If the agent has tested $x = 0$, $x = 1$, and $x = 2$ in a function-identification task, the set of all three observations matters; the current observation $f(2)$ by itself is insufficient.

Memory need not be a verbatim history. It can be a compact statistic, a database, a summary, a learned vector representation, a probability distribution over hypotheses, or a structured collection of past events.

4.6 Policy

The policy is the decision rule. In a simple deterministic form, $a_t = \pi(M_t, o_t)$.

A stochastic policy instead defines a distribution $\pi(a \mid M_t, o_t)$ and samples the next action from that distribution.

The policy is the part of the system that answers the question: *given what I currently know, what should I do next?*

In a classical system, π may be a hand-written rule, a planning algorithm, a dynamic-programming solution, or a learned neural network. In an LLM-based agent, an LLM may implement part of π, but the policy can still be constrained and validated by deterministic code.

4.7 Objective

An agent needs a criterion for preferring one trajectory over another. We may assign an immediate reward r_t, a cost c_t, or a final task loss.

A common objective is to maximize cumulative reward $\sum_{t=0}^{T} \gamma^t r_t$, where $0 < \gamma \leq 1$ discounts future rewards. For many practical agents it is more natural to minimize a task-specific cost such as J = prediction error + λ tool cost.

The coefficient λ controls the trade-off between task quality and resource consumption.

An agent without an objective cannot meaningfully distinguish a good action from a bad one.

4.8 Stopping Rule

Many agents should not act indefinitely. Let τ denote a stopping rule. The agent terminates when $\tau(M_t, o_t) = 1$.

Examples include stopping when confidence exceeds 0.95, when the goal has been reached, when no affordable tool remains, when a maximum number of actions has been used, or when further expected improvement is too small.

The stopping decision is itself part of intelligent behaviour.

A Compact Mathematical Description

A useful abstraction for this course is to describe an agent by

$$\mathscr{A} = (\mathcal{S}, \mathcal{O}, \mathcal{A}, M, \pi, U, J, \tau),$$

together with an environment that specifies how states evolve and observations are produced.

This notation should not be interpreted as a universal formal definition used identically by all fields. Its purpose is to force us to identify the important components of an agent explicitly.

For any new problem, students should immediately ask:

1. What is the true environment state?
2. What can the agent actually observe?
3. What actions are available?
4. What information must be remembered?
5. What is the decision policy?
6. What is the objective?
7. When does the agent stop?

If these questions cannot be answered clearly, the agent design is probably underspecified.

The Agent Loop

A generic first-principles agent can be written conceptually as:

```
memory = initial_memory()

while True:
    observation = observe_environment()
    memory = update_memory(memory, observation)
    action = choose_action(memory)
    if action == STOP:
        break
    validated_action = validate(action)
    execute(validated_action)
```

The apparent simplicity of this loop is important. Modern frameworks add abstractions for persistence, tool registration, message routing, retries, graphs, asynchronous execution, tracing, and distributed agents, but the conceptual core remains close to this loop.

A particularly useful interpretation is

$$\boxed{\pi : \text{information available now} \longrightarrow \text{next action.}}$$

The intelligence of the system lies partly in how the information state is represented and partly in how the next action is selected.

Open-Loop and Closed-Loop Behaviour

Suppose a program generates an entire sequence of actions $(a_1, \ldots, a_T)$ before seeing any intermediate outcomes. This is an *open-loop* plan.

A closed-loop agent instead chooses a_{t+1} after observing the consequences of a_t. Thus a_{t+1} can adapt to unexpected events.

For example, an open-loop diagnostic script might always perform five tests in a fixed order. A closed-loop diagnostic agent may stop after two tests if one hypothesis has become overwhelming, or may choose a different third test depending on the first two results.

Closed-loop behaviour is one of the main sources of flexibility in agentic systems.

Observability

8.1 Fully Observable Environments

An environment is fully observable when the current observation contains all task-relevant information about the current state.

If a chess program is given the complete board configuration, then the board state is fully visible. The program does not need to infer hidden piece locations.

In such a setting, a policy may often be written simply as $a_t = \pi(s_t)$.

8.2 Partially Observable Environments

An environment is partially observable when the agent cannot directly determine the true state. It must combine current observations with memory or inference.

Suppose a hidden fault H can take one of five values. A diagnostic test produces either `NORMAL` or `ABNORMAL`. One observation does not reveal H exactly. The agent therefore maintains a belief over possibilities.

If $b_t(h)$ denotes the agent's current probability that the hidden state is h, then b_t acts as an internal information state. Later lectures will formalize belief updates using Bayes' rule.

Partial observability is ubiquitous in real systems: sensors are noisy, APIs expose only limited information, documents are incomplete, users omit context, and other agents may possess private information.

Deterministic and Stochastic Environments

A deterministic environment gives the same outcome whenever the same action is taken in the same state. The noiseless function-identification problem is deterministic: if the hidden function is $f(x) = 2x + 3$, then every query at $x = 4$ returns 11.

A stochastic environment contains randomness. If an experimental instrument returns $f(x) + \varepsilon$, where ε is noise, identical queries may return different values. Similarly, a network request may succeed or fail probabilistically.

The distinction matters because deterministic agents can often reason about exact consequences, while stochastic environments require reasoning about distributions, uncertainty, expected outcomes, confidence, and risk.

Other Useful Environment Properties

The classical agent literature often classifies environments along several dimensions. These distinctions remain useful for modern LLM agents.

10.1 Episodic versus Sequential

In an episodic task, each decision is effectively independent of previous decisions. Image classification is close to episodic: classifying one image usually does not change the next image.

In a sequential task, an action changes future possibilities. Navigation, coding, diagnosis, negotiation, and tool-using agents are sequential.

Agentic systems are predominantly sequential.

10.2 Static versus Dynamic

A static environment does not change while the agent is deliberating. A dynamic environment may change independently of the agent.

A stored dataset is mostly static. A financial market, multiplayer game, live network, or collaborative workspace is dynamic.

10.3 Discrete versus Continuous

Actions, observations, states, and time may be discrete or continuous. A grid world has discrete positions and movements. Robot control may involve continuous position, velocity, torque, and time.

10.4 Known versus Unknown

In a known environment, the agent knows the relevant transition and observation mechanisms. In an unknown environment, the agent must learn or estimate them while interacting.

The unknown-function scientist agent has a known family of candidate functions but unknown coefficients. If the candidate family itself were unknown, the problem would become more difficult.

Rationality Is Not Omniscience

A rational agent chooses actions that are good given the information available to it. Rationality does not require knowing the true hidden state.

Suppose an agent has observed evidence strongly supporting hypothesis H_1 and acts accordingly, but the true state happens to be the less likely H_2. The action may still have been rational if it maximized expected performance under the agent's information.

This distinction is important because agents operate under uncertainty. We should evaluate whether their decisions were justified by available evidence, not demand impossible foreknowledge.

Similarly, a rational agent may deliberately gather information before acting. If two hypotheses imply different optimal actions, then paying a small cost for a diagnostic observation can have positive value.

Reactive, Stateful, Model-Based, and Planning Agents

These categories are best viewed as increasing levels of internal structure rather than rigidly separate species.

12.1 Reactive Agent

A reactive agent chooses its action directly from the current observation, $a_t = \pi(o_t)$.

Example: if an obstacle sensor reports `LEFT_BLOCKED`, turn right.

Reactive agents are simple and fast, but they fail when the present observation is insufficient.

12.2 Stateful Agent

A stateful agent maintains memory M_t and chooses $a_t = \pi(M_t, o_t)$.

Example: a diagnostic system remembers which tests have already been performed and their outcomes.

12.3 Model-Based Agent

A model-based agent maintains an internal model of the environment or of hidden hypotheses. It predicts how actions may affect future states or what observations different hypotheses would produce.

The scientist agent is model-based because it maintains candidate mathematical functions.

12.4 Planning Agent

A planning agent evaluates sequences of possible future actions before choosing the next action.

Instead of asking only "What is the best action now?", it asks "If I take this action, what states may follow, and what could I do afterward?"

Planning therefore introduces look-ahead.

Tools as Actions

In modern agentic systems, actions frequently take the form of tool calls. Examples include querying a database, searching documents, executing Python, calling an API, reading or writing a file, sending a message, invoking a mathematical solver, or running a diagnostic test.

A tool can be represented abstractly as a function $T : \mathcal{X} \to \mathcal{Y}$, but an agentic tool has additional operational meaning: the agent decides whether to call it, chooses the arguments, receives its output as an observation, and may modify its future behaviour accordingly.

This is why simply calling an API does not by itself make a program an agent. The important issue is whether tool use occurs inside an adaptive decision loop.

Agent Memory

Memory deserves careful treatment because the word is used loosely in modern systems.

At the most elementary level, memory is any internal variable that allows future decisions to depend on past interaction.

For the scientist agent, $M_t = \{(x_1, y_1), \dots, (x_t, y_t)\}$ is sufficient memory.

For a navigation agent, memory might be a discovered map. For a diagnostic agent, memory might include test outcomes together with posterior probabilities over faults.

Memory can therefore be raw interaction history, a compressed summary, a set of sufficient statistics, a probability distribution over hidden states, a structured knowledge store, or a database of previous episodes.

A central design question is not "Does the agent have memory?" but rather "What information from the past is useful for choosing future actions?"

Agent State versus Environment State

Students often confuse these two concepts.

The environment state s_t describes the actual world. The agent's internal state M_t describes what the agent currently knows or remembers.

Suppose the hidden function is $f(x) = 2x^2 - 3x + 5$. The coefficients $(2, -3, 5)$ are part of the environment state. Before any experiments, the agent does not know them. After observing $f(0) = 5$, $f(1) = 4$, and $f(2) = 7$, its memory contains these observations and it can infer the coefficients.

Thus, the environment state can remain unchanged while the agent's information state changes substantially.

This separation is essential for understanding uncertainty, information gathering, and belief-state methods.

Agent, Workflow, Chatbot, and RAG: Important Distinctions

These terms overlap in practice, but conceptually they are different.

16.1 Fixed Workflow

A fixed workflow follows a predetermined sequence such as $A \to B \to C \to D$. The sequence is known in advance.

A workflow can include branches, but if every branch is explicitly prescribed by the programmer, most of the decision structure remains fixed.

16.2 Agent

An agent dynamically chooses actions based on observations and internal state. Its trajectory may differ substantially between runs.

Thus, the relevant distinction is not whether the implementation contains a loop, but where the decision about the next action comes from.

16.3 Chatbot

A chatbot maps conversation context to text responses. A chatbot becomes more agent-like when it can choose and execute external actions, observe their outcomes, maintain persistent state, and continue acting toward an objective.

16.4 Retrieval-Augmented Generation

A standard RAG pipeline retrieves information and supplies it to a language model. If retrieval occurs in a fixed predetermined way, the system is primarily a pipeline.

An agentic retrieval system may decide whether retrieval is necessary, what query to issue, whether the retrieved evidence is sufficient, whether to reformulate the query, and when to stop.

Thus, *RAG is a mechanism for obtaining information*; agency concerns the adaptive control of actions.

Running Example: The Unknown-Function Scientist Agent

We now formalize the first programming problem of the course.

The environment secretly selects either a linear function $f(x) = ax + b$ or a quadratic function $f(x) = ax^2 + bx + c$. The coefficients are unknown to the agent.

The agent can perform experiments. An action may be $a_t = \texttt{experiment}(x_t)$, and the observation is $o_{t+1} = f(x_t)$.

The agent stores all observed pairs in $M_t = \{(x_1, y_1), \dots, (x_t, y_t)\}$.

It may also perform internal actions such as fitting a linear hypothesis, fitting a quadratic hypothesis, validating a hypothesis, or stopping.

17.1 Environment State

The environment state contains the hidden function family, its coefficients, and possibly the number of queries already used. For a quadratic example, the hidden component might be $f(x) = 2x^2 - 3x + 5$.

17.2 Observation

If the agent chooses $\texttt{experiment}(2)$, it observes $f(2) = 7$. The observation reveals one consequence of the hidden function, not the function itself.

17.3 Memory

After querying $x = 0, 1, 2$, memory may be $M_3 = \{(0, 5), (1, 4), (2, 7)\}$.

17.4 Inference

For equally spaced points $0, 1, 2$, define the second finite difference as $y_2 - 2y_1 + y_0$.

If $f(x) = ax + b$, the second finite difference is 0. If $f(x) = ax^2 + bx + c$, the second finite difference is $2a$.

For the observations $5, 4, 7$, we obtain $7 - 2(4) + 5 = 4$, hence $2a = 4$ and $a = 2$. Since $f(0) = c = 5$, and $f(1) = a + b + c = 4$, we get $b = -3$. Therefore the candidate model is $f(x) = 2x^2 - 3x + 5$.

Notice the structure: observations are converted into memory; memory supports a hypothesis; the hypothesis affects the next action.

17.5 Validation as an Action

The agent should not automatically trust its first hypothesis. It can query a new point, say $x = 3$. Its model predicts 14. If the environment returns 14, the model gains support.

The validation action is valuable because it tests a consequence that was not used to fit the model.

17.6 Stopping

The agent may stop after successful validation or when the experiment budget is exhausted.

The stopping rule therefore expresses a trade-off: more experiments may improve confidence, but experiments have a cost.

17.7 Objective

One possible loss is $J = \text{MSE}_{\text{test}} + \lambda N_{\text{queries}}$. The first term measures predictive error on unseen points; the second penalizes excessive experimentation.

This objective prevents a trivial strategy of performing arbitrarily many experiments.

Why the Scientist Problem Is Genuinely Agentic

It is tempting to say that this is merely polynomial fitting. That would be true if the complete dataset were supplied in advance.

The agentic version is different because the system itself decides which x to query, when enough information has been collected, which model family to consider, whether the current hypothesis should be validated, whether validation failure should trigger revision, and when to stop.

Thus the agent controls its own data-acquisition process.

This introduces an important concept that will recur throughout the course: *an action may be valuable because of the information it is expected to produce.*

A Second Example: Partially Known Grid Navigation

Consider a robot on a 10×10 grid. The true map contains obstacles, but initially the robot knows only its starting position and goal.

The environment state contains the complete map and robot position. The observation may reveal only nearby cells. The actions are `UP`, `DOWN`, `LEFT`, `RIGHT`, and `SCAN`.

The agent's memory contains its discovered map. A movement action changes the robot's location; a scan action primarily gathers information.

Suppose the objective is $J = N_{\text{moves}} + 2N_{\text{scans}} + 5N_{\text{collisions}}$, subject to eventually reaching the goal.

This example illustrates several ideas absent from ordinary path finding. The true map is hidden, actions reveal information, the agent must update its internal map, and a previously planned path may become invalid after a newly discovered obstacle.

Decision Quality and Resource Cost

Real agents are rarely judged only by whether they eventually succeed.

Relevant metrics may include task success, number of actions, tool cost, wall-clock latency, energy or computation, number of model calls, robustness across repeated runs, safety violations, unnecessary actions, and calibration of confidence.

A general design principle is therefore to make performance multidimensional.

If two agents achieve the same accuracy, the one using fewer expensive tool calls may be preferable. If two agents use equal cost, the one with greater robustness may be preferable.

This becomes especially important when LLM calls, web searches, code execution, or external services have non-negligible cost.

Failure Modes Already Visible at This Stage

Even simple agents can fail in qualitatively different ways.

21.1 Perception Failure

The observation is incorrect, incomplete, noisy, or misinterpreted.

21.2 Memory Failure

Relevant information is forgotten, corrupted, duplicated, or summarized incorrectly.

21.3 Decision Failure

The agent has enough information but selects a poor action.

21.4 Model Failure

The internal model of the world is wrong. The scientist agent may assume the function is quadratic when the true function lies outside the permitted family.

21.5 Tool Failure

A tool may return an error, timeout, malformed result, or misleading observation.

21.6 Stopping Failure

The agent may stop too early or continue acting after sufficient evidence has already been obtained.

21.7 Objective Failure

The system may optimize the specified metric while behaving undesirably because the metric does not fully capture the intended goal.

These categories will later help us build systematic failure taxonomies.

Where an LLM Will Enter Later

Nothing in the formal agent loop requires an LLM.

In later lectures, an LLM may be used to implement part of the policy. Instead of a hand-written function such as $\pi(M_t) = \texttt{check_disk}$ under one condition and $\pi(M_t) = \texttt{check_memory}$ under another, we may provide the current memory and available tools to an LLM and ask it to propose the next action.

However, the surrounding system should still control important invariants. The LLM may propose `check_disk`, while Python verifies that the tool exists, has not already been used, lies within budget, and is safe to execute.

This motivates the architectural separation

$$\boxed{\text{LLM proposes} \longrightarrow \text{controller validates} \longrightarrow \text{environment executes.}}$$

Understanding this separation is essential before studying tool calling or agent frameworks.

A Dynamical-Systems View of an Agent

There is another useful perspective. The combined environment and agent form a coupled dynamical system.

The environment evolves according to the current state and chosen action, while the agent's memory evolves according to observations. In a deterministic setting we may write $s_{t+1} = F(s_t, a_t)$, $o_{t+1} = G(s_{t+1})$, $M_{t+1} = U(M_t, o_{t+1})$, and $a_{t+1} = \pi(M_{t+1})$.

The important point is that the action generated by the agent feeds back into the environment, whose next observation feeds back into the agent.

This is precisely why "closed loop" is a useful phrase: the agent is not merely processing an input stream; it partly determines the future inputs it will receive.

The Information-State View

In many problems, the true state s_t is hidden. The agent therefore acts using an information state I_t.

The information state may be the complete history h_t, a compressed memory M_t, or a belief distribution over hidden states.

If I_t contains everything from the past that is relevant for future optimal decision making, then the policy can be written simply as $a_t = \pi(I_t)$.

This idea is powerful because it separates two questions:

1. How should the agent summarize what it currently knows?
2. Given that summary, how should it choose the next action?

Later lectures on memory, Bayesian beliefs, context engineering, and LLM prompting can all be understood through this separation.

What Makes an Agent "Autonomous"?

Autonomy is not an all-or-nothing property.

A system has greater autonomy when it makes more of the following decisions itself: what subgoal to pursue next, which tool to call, what arguments to pass to the tool, whether a result is sufficient, whether to revise a plan, whether to ask for human input, and when to stop.

A fixed script may have low autonomy. An unconstrained long-running agent may have high autonomy.

Greater autonomy can increase flexibility, but it also increases the need for evaluation, permissions, monitoring, and safety mechanisms. Thus, "more autonomous" does not automatically mean "better."

A Practical Design Checklist

Whenever you encounter a proposed agentic system, write down the following before discussing frameworks or implementation details:

1. **Goal:** What exactly is the system trying to achieve?
2. **Environment:** What world does it interact with?
3. **Hidden state:** What relevant facts are not directly visible?
4. **Observations:** What information can the system receive?
5. **Actions:** What can it actually do?
6. **Tools:** Which actions are implemented through external functions or services?
7. **Memory:** What must be retained from previous interactions?
8. **Policy:** What determines the next action?
9. **Objective:** How is success or failure measured?
10. **Constraints:** What actions are forbidden, costly, or budget-limited?
11. **Stopping rule:** When should interaction terminate?
12. **Evaluation:** How will we know whether the agent is actually good?

This checklist will be reused throughout the course.

Worked Classification Examples

27.1 Calculator

A calculator receiving 23×47 and returning 1081 is not meaningfully agentic in this course. There is no sequential interaction or adaptive choice of actions.

27.2 Fixed RAG Pipeline

A system that always retrieves the top five documents for a query and then asks an LLM to answer is primarily a pipeline. Retrieval is predetermined.

27.3 Agentic RAG

A system that decides whether to search, formulates a query, examines the results, decides whether evidence is sufficient, reformulates the query if necessary, and stops when confidence is adequate exhibits agentic control.

27.4 Coding Agent

A coding agent may inspect files, edit code, run tests, observe failures, revise its hypothesis, modify code again, and stop only after validation. This is clearly closed-loop behaviour.

27.5 Server Diagnostic Agent

A server diagnostic agent chooses which test to perform next based on previous results. Some actions gather information rather than directly fixing the server. This is a canonical partially observable agent problem.

Conceptual Questions

Students should be able to discuss the following without using software frameworks.

1. Can a deterministic program be an agent? Explain.
2. Can a system use an LLM without being agentic?
3. Can a system be agentic without using an LLM?
4. Why is the environment state different from the agent's memory?
5. Why might a rational agent perform an action that does not immediately improve the external state?
6. When does a fixed workflow become an adaptive agent?
7. Why should stopping be considered part of the policy?
8. What makes an observation valuable?
9. Why is more autonomy not automatically desirable?
10. Which parts of an agent should remain under deterministic program control even when an LLM is used?

Exercises

Exercise 1: Identify the Components

For each of the following systems, identify $\mathcal{S}$, $\mathcal{O}$, $\mathcal{A}$, M_t, π, the objective, and the stopping rule:

1. a robot finding an exit from an unknown maze;

2. a medical triage assistant asking questions before recommending a level of urgency;
3. a code-debugging agent with access to a compiler and test suite;
4. a research assistant that searches papers until it has sufficient evidence for a claim.

Exercise 2: Open Loop or Closed Loop?

A travel program first generates ten web searches, then executes all ten, and finally summarizes the results. Compare it with a system that executes one search, inspects the result, and decides the next search dynamically. Which is more agentic, and why?

Exercise 3: Memory Sufficiency

Suppose an agent is trying to identify whether a hidden coin has bias $p = 0.3$ or $p = 0.7$. Is it necessary to store the complete sequence of heads and tails? Propose a smaller memory state sufficient for inference.

Exercise 4: Scientist Agent

For a hidden quadratic $f(x) = 3x^2 - 2x + 4$, compute $f(0)$, $f(1)$, and $f(2)$. Use the second finite difference to recover the quadratic coefficient, then recover the remaining coefficients.

Exercise 5: Cost of Information

Suppose one diagnostic test costs 1 unit and another costs 10 units. The expensive test is more informative. Explain why "choose the most informative test" is not always a rational policy. What quantity should be compared instead?

Lecture Summary

The central message of this lecture is that an agent is best understood as a system participating in a sequential closed loop with an environment. At each time step, the environment has a true state s_t, the agent receives an observation o_t, updates memory M_t, and chooses an action a_t using a policy π. The action changes the future interaction, and the process continues until a stopping rule terminates it.

The most important distinctions are true state versus observed information, current observation versus accumulated memory, fixed workflow versus adaptive decision making, acting to change the world versus acting to acquire information, task success versus resource-aware performance, and an LLM as a component of a policy versus an LLM as the entire system.

A compact conceptual model to retain is

$$\text{Agent} = \text{information state} + \text{decision policy} + \text{actions/tools}$$

$$+\text{feedback from an environment} + \text{objective} + \text{stopping rule}.$$

The rest of the course will progressively enrich each component. Lecture 2 will introduce LLMs as decision-making components inside this architecture rather than treating them as complete agents by themselves.